

Version control

Introduction

- Version control, also known as source control or revision control, in software engineering refers to the practice and tools used to track and manage changes to source code and other project files over time.
- It is a fundamental component of modern software development, particularly in collaborative environments.

Key Aspects of version control

- **Tracking Changes:**
- Version control systems (VCS) record every modification made to files, including who made the change, when it was made, and what exactly was altered. This creates a complete history of the project.
- Collaboration:
- VCS enables multiple developers to work on the same codebase concurrently without overwriting each other's work. It provides mechanisms for merging changes from different contributors and resolving conflicts.

- Reversion and Rollback:
- If a mistake is introduced or a feature needs to be removed, version control allows developers to easily revert to a previous, stable version of the code or specific files.
- Branching and Merging:
- Developers can create separate "branches" to work on new features or bug fixes in isolation from the main development line. Once the work is complete and tested, these branches can be "merged" back into the main codebase.
- Backup and Recovery: The version history stored in the VCS acts as a comprehensive backup, protecting against data loss due to hardware failure, accidental deletion, or other unforeseen events.

Common types of Version Control Systems:

- Centralized Version Control Systems (CVCS):
- These systems use a single, central server to store all versions of the code. Examples include Subversion (SVN) and Perforce.
- Distributed Version Control Systems (DVCS):
- In DVCS, each developer has a complete copy of the repository, including the full history. This allows for offline work and more flexible workflows. Git is the most prominent example of a DVCS.

Developers can create separate "branches" to work on new features or bug fixes in isolation from the main development line. Once the work is complete and tested, these branches can be merged back into the main line. If a mistake is introduced to a feature, it can be removed, version control allows developers to easily revert to a previous, stable version of the code or specific files.

Developers can create separate "branches" to work on new features or bug fixes in isolation from the main development line. Once the work is complete and tested, these branches can be merged back into the main line. If a mistake is introduced to a feature, it can be removed, version control allows developers to easily revert to a previous, stable version of the code or specific files.

The version history stored in the VCS acts as a comprehensive backup, protecting against data loss due to hardware failure, accidental deletion, or other unforeseen events.

Developers can create separate "branches" to work on new features or bug fixes in isolation from the main development line. Once the work is complete and tested, these branches can be merged back into the main line. If a mistake is introduced to a feature, it can be removed, version control allows developers to easily revert to a previous, stable version of the code or specific files.

The version history stored in the VCS acts as a comprehensive backup, protecting against data loss due to hardware failure, accidental deletion, or other unforeseen events.

Backup and Recovery.